

《计算拓扑数据分析》第三章笔记：持续算法

“基于持续同调的拓扑特征识别及其应用” SRTP项目组

目录

章节概览	2
1 瓶颈距离的计算	2
1.1 问题转化	2
1.2 算法思路	2
2 持续配对算法 (PAIRPERSISTENCE)	3
2.1 基本观察	3
2.2 配对规则	3
2.3 算法描述	3
3 矩阵约简算法 (MATPERSISTENCE)	4
3.1 过滤边界矩阵	4
3.2 约简矩阵与低行索引	4
3.3 核心算法	4
3.4 配对定理	4
3.5 复杂度分析	5
4 高效实现技巧	5
4.1 清除策略 (Clearing)	5
4.2 扭曲矩阵 (上同调算法)	5
算法总结	6
5 零维持续同调算法 (ZEROPERDG)	6
5.1 核心观察	6
5.2 三种情况	6
5.3 配对规则	7
5.4 算法描述	7
5.5 复杂度分析	8
5.6 与最小生成树的联系	8
5.7 图滤过情形	8

章节概览

本章聚焦于持续同调的计算方面。给定一个单纯滤过，我们的目标是计算其持续图。这等价于找出所有导致同调类出生与死亡的单纯形配对。我们将依次介绍：

1. 瓶颈距离的计算 (**BOTTLENECK 算法**)：通过二分图完美匹配计算两个持续图之间的距离；
2. 配对算法 (**PAIRPERSISTENCE 算法**)：组合视角下的持续配对算法；
3. 矩阵约简算法 (**MATPERSISTENCE 算法**)：持续同调计算的核心算法，通过边界矩阵的列消元计算持续配对；
4. 高效实现技巧 (**CLEARPERSISTENCE 算法**)：清除策略与上同调对偶优化；
5. 零维持续同调算法 (**ZEROPERDGD 算法**)：基于并查集的零维持续同调专用算法。

1 瓶颈距离的计算

在 3.2.1 节中我们定义了瓶颈距离，现在考虑其计算问题。

1.1 问题转化

给定两个持续图 $\text{Dgm}_p(\mathcal{F}_f)$ 和 $\text{Dgm}_p(\mathcal{F}_g)$ ，设 A 和 B 分别为其非对角点集。为处理两个图点数不同的情况，我们将每个点投影到对角线上：

- $\bar{a}$: 点 $a \in A$ 在对角线 Δ 上的最近点；
- $\bar{b}$: 点 $b \in B$ 在对角线 Δ 上的最近点；
- 构造扩展点集: $\tilde{A} = A \cup \bar{B}$, $\tilde{B} = B \cup \bar{A}$ 。

则瓶颈距离等于：

$$d_b = \min_{\pi} \max_{a \in \tilde{A}} \|a - \pi(a)\|_{\infty},$$

其中 π 遍历 $\tilde{A}$ 到 $\tilde{B}$ 的所有双射。

1.2 算法思路

算法 1.1 (BOTTLENECK). 1. 计算所有可能的 $O(n^2)$ 个点对 L_{∞} 距离，排序得 $\delta_0 \leq \delta_1 \leq \dots \leq \delta_{n'}$ ；

2. 二分搜索最小的 δ ，使得存在一个完美匹配；

3. 对给定的 δ ，构造二分图 $G = (\tilde{A} \cup \tilde{B}, E)$ ，其中 $(a, b) \in E$ 当且仅当 $\|a - b\|_{\infty} \leq \delta$ ；

4. 检查是否存在完美匹配 (*Hopcroft-Karp* 算法 $O(n^{2.5})$ ，利用几何结构可优化至 $O(n^{1.5})$)；

5. 若存在，减小 δ ；否则增大 δ 。

注记 1.2. **BOTTLENECK 算法** 的复杂度为 $O(n^{1.5} \log n)$ ，其中 n 是持续图中非对角点的数量。为避免排序所有 $O(n^2)$ 个距离，可利用点的几何嵌入直接进行二分搜索。

2 持续配对算法 (PAIRPERSISTENCE)

2.1 基本观察

我们假设输入是逐单形滤过:

$$\emptyset = K_0 \hookrightarrow K_1 \hookrightarrow \cdots \hookrightarrow K_n = K,$$

其中 $K_j \setminus K_{j-1} = \sigma_j$ 为单个单纯形。

事实 2.1 (添加单纯形时的两种可能). 添加一个 p -单纯形 σ_j 时, 恰好发生以下两种情况之一:

1. **正单纯形 (Positive)**: 创建一个非边界的 p -闭链, β_p 增加 1;
2. **负单纯形 (Negative)**: 销毁一个已有的 $(p-1)$ -闭链, β_{p-1} 减少 1。

注记 2.2. 正单纯形也称为“创建者 (creator)”, 负单纯形也称为“销毁者 (destructor)”。每个负单纯形通过“最年轻优先”规则唯一配对一个正单纯形。

2.2 配对规则

当一个 p -单纯形 σ_j 被添加时, 检查其边界 $\partial\sigma_j$ 是否是一个边界闭链 (即已在当前同调群中为零):

- 若 $\partial\sigma_j$ 已是边界, 则 σ_j 创建了一个新的 p -维同调类 (正单纯形);
- 若 $\partial\sigma_j$ 不是边界, 则 σ_j 销毁了由某个更早的 $(p-1)$ -单纯形创建的类 (负单纯形)。

为找到被销毁类的出生点, 我们从 $\partial\sigma_j$ 中取出最年轻的未配对的 $(p-1)$ -单纯形——这就是“最年轻优先”原则。

2.3 算法描述

算法 2.3 (PAIRPERSISTENCE). **输入**: 逐单形滤过的单纯形序列 $\sigma_1, \sigma_2, \dots, \sigma_n$

输出: 每个单纯形的正/负标记及持续配对

1. 对 $j = 1$ 到 n :
 - (a) 令 $c := \partial_p \sigma_j$;
 - (b) 令 σ_i 为 c 中最年轻的未配对的 $(p-1)$ -单纯形;
 - (c) 当 σ_i 已配对且 c 非空时:
 - i. 令 c' 为与 σ_i 配对的单纯形所销毁的闭链;
 - ii. $c := c' + c$ (模 2 加法);
 - iii. 更新 σ_i 为 c 中最年轻的未配对的 $(p-1)$ -单纯形;
 - (d) 若 c 非空: σ_j 为负单纯形, 生成配对 (σ_i, σ_j) ;
 - (e) 否则: σ_j 为正单纯形。

3 矩阵约简算法 (MATPERSISTENCE)

3.1 过滤边界矩阵

定义 3.1 (过滤边界矩阵). 设滤过 $\emptyset = K_0 \hookrightarrow K_1 \hookrightarrow \dots \hookrightarrow K_n = K$, 单纯形按滤过顺序排列为 $\sigma_1, \dots, \sigma_n$. 过滤边界矩阵 D 是 $n \times n$ 矩阵, 其中

$$D[i, j] = \begin{cases} 1, & \sigma_i \in \partial\sigma_j, \\ 0, & \text{否则.} \end{cases}$$

即第 j 列表示 σ_j 的边界 (在 $\mathbb{Z}_2$ 系数下)。

注记 3.2. 因 $\partial^2 = 0$, 边界矩阵满足 $D^2 = 0$ (在适当的意义下)。

3.2 约简矩阵与低行索引

定义 3.3 (低行索引). 对矩阵 A , $\text{low}_A[j]$ 表示第 j 列中最后一个 1 所在的行索引 (即最大的非零行索引)。若列为空, 则 $\text{low}_A[j] = -1$ 。

定义 3.4 (约简矩阵). 矩阵 A 称为约简的 (*reduced*), 若对任意 $j \neq j'$, $\text{low}_A[j] \neq \text{low}_A[j']$, 即所有非空列的低行索引互不相同。

事实 3.5. 约简矩阵的非零列在 $\mathbb{Z}_2$ 上线性无关。

3.3 核心算法

MATPERSISTENCE 算法通过从左到右的列加法将过滤边界矩阵约简为约简形式。列加法 $\text{col}_j \leftarrow \text{col}_j + \text{col}_i$ ($i < j$) 对应于将第 i 列加到第 j 列。

算法 3.6 (MATPERSISTENCE). **输入:** 过滤边界矩阵 D (列和行按滤过顺序排列)

输出: 约简矩阵, 其中每列要么为空, 要么有唯一的 low 值

1. 对 $j = 1$ 到 $|D|$:

(a) 当存在 $j' < j$ 使得 $\text{low}[j'] = \text{low}[j]$ 且 $\text{low}[j] \neq -1$:

i. $\text{col}[j] \leftarrow \text{col}[j] + \text{col}[j']$

(b) 若 $\text{low}[j] \neq -1$, 则生成配对 $(\sigma_{\text{low}[j]}, \sigma_j)$

注记 3.7. 算法的核心逻辑是: 当两列的低行索引冲突时, 将左侧列加到右侧列上, 直到冲突消除。这本质上是在矩阵上执行“最年轻优先”配对。

3.4 配对定理

定理 3.8 (持续配对判定定理). 设 D 为过滤边界矩阵, $R = DV$ 为 D 的约简形式 (V 为上三角矩阵)。则

$$\sigma_i \text{ 与 } \sigma_j \text{ 形成持续配对} \iff \text{low}_R[j] = i.$$

注记 3.9. 该定理表明, 持续配对完全由约简矩阵中的低行索引决定, 且与具体的约简过程无关。

3.5 复杂度分析

- **MATPERSISTENCE 算法**: $O(n^3)$, 其中 n 为单纯形总数;
- 利用稀疏矩阵表示可优化;
- 使用快速矩阵乘法可达到 $O(n^\omega)$, 其中 $\omega \in [2, 2.373)$ 为矩阵乘法指数。

4 高效实现技巧

4.1 清除策略 (Clearing)

注记 4.1 (核心观察). 若一个 p -单纯形 σ_j 是正单纯形, 它将在更高维度的处理中与某个 $(p+1)$ -单纯形配对。因此在处理 p 维边界矩阵时, 可跳过这些已知的正单纯形, 不必将其约简为零列。

算法 4.2 (CLEARPERSISTENCE). **输入**: 按维度排列的边界矩阵 $D_1, D_2, \dots, D_d$

输出: 约简矩阵, 每个负单纯形的列具有唯一的低行索引

1. **MATPERSISTENCE**(D_d);
2. 对 $i = d-1$ 到 1:
 - (a) 对 D_i 的每一列 j :
 - i. 若 σ_j 在处理 D_{i+1} 时已配对, 则跳过该列;
 - ii. 否则, 执行与 **MATPERSISTENCE** 相同的列约简;
 - iii. 若约简后非空, 生成配对。

注记 4.3. 清除策略在实际计算中 (特别是 *Rips* 滤过) 能大幅提升效率, 因为高维正单纯形的数量通常远大于负单纯形。

4.2 扭曲矩阵 (上同调算法)

定义 4.4 (扭曲矩阵). 矩阵 D_p 的**扭曲矩阵** D_p^* 定义为 D_p 的“反转置”(先转置, 再反转行列顺序):

$$D_p^*(i, j) = D_p(n+1-j, m+1-i).$$

命题 4.5 (扭曲矩阵的对偶性). 在 D_p 中配对 (σ^{p-1}, σ^p) 当且仅当在 D_p^* 中配对 (σ^p, σ^{p-1}) 。

注记 4.6. 这一对偶性允许我们选择计算效率更高的方向: 若高维单纯形较多, 使用扭曲矩阵从低维往高维处理可能更优。

算法总结

算法	输入	输出	核心理念
BOTTLENECK	两个持续图	瓶颈距离	二分图完美匹配 + 二分搜索
PAIRPERSISTENCE	单纯形序列	持续配对	最年轻优先原则
MATPERSISTENCE	过滤边界矩阵	约简矩阵 + 配对	从左到右列消元
CLEARPERSISTENCE	各维边界矩阵	约简矩阵	跳过已知正单纯形
ZEROPERDG	1-复形 + 顶点函数	零维持续配对	并查集维护连通分量

5 零维持续同调算法 (ZEROPERDG)

对于一般持续同调算法, 已知最优运行时间为 $O(n^\omega)$, 其中 n 是滤过中单纯形的总数。然而, 对于 **零维持续同调** (即持续图 $\text{Dgm}_0 f$), 存在更为高效的专用算法。

5.1 核心观察

计算 $\text{Dgm}_0 f$ 仅需复形 K 的 **1-骨架** (即顶点集 V 和边集 E), 因为零维同调仅关心连通分量的变化。

设顶点按函数值非递减排序:

$$f(v_1) \leq f(v_2) \leq \cdots \leq f(v_n).$$

令 K_i 为前 i 个顶点的下星之并, 即 $K_i = \bigcup_{j \leq i} \overline{\text{Lst}}(v_j)$ 。零维同调 $H_0(K_i)$ 的本质信息就是 K_i 中连通分量的数量与组成。

5.2 三种情况

当处理顶点 v_j 时, 考察其下星 $\text{Lst}(v_j)$:

- 情况 1:** $\text{Lst}(v_j) = \{v_j\}$ 。此时 v_j 在 K_j 中创建一个新的连通分量, v_j 为创建者。
- 情况 2:** $\text{Lst}(v_j)$ 中的所有边都连接到 K_{j-1} 中的同一个连通分量 C 。此时 C 扩展包含 v_j 及其关联边, 但 $H_0(K_{j-1}) \cong H_0(K_j)$, 无拓扑变化。
- 情况 3:** $\text{Lst}(v_j)$ 中的边连接到 K_{j-1} 中的多个连通分量 $C_1, C_2, \dots, C_r$ ($r \geq 2$)。此时这些分量合并为一个新分量:

$$C' = C_1 \cup C_2 \cup \cdots \cup C_r \cup \text{Lst}(v_j).$$

包含映射 $K_{j-1} \hookrightarrow K_j$ 诱导一个满射同态 $\xi: H_0(K_{j-1}) \rightarrow H_0(K_j)$, 且

$$\beta_0(K_j) = \beta_0(K_{j-1}) - (r - 1),$$

即 $r - 1$ 个分量被销毁, 仅一个存活为 C' 。

5.3 配对规则

命题 5.1 (Proposition 3.17). 设情况 3 发生, $\text{Lst}(v_j)$ 中的边合并了 K_{j-1} 中的分量 $C_1, \dots, C_r$ 。令 v_{k_i} 为分量 C_i 中的全局最小值顶点 (即 $f(v_{k_i})$ 最小), 并设

$$f(v_{k_1}) \leq f(v_{k_2}) \leq \dots \leq f(v_{k_r}).$$

则顶点 v_j 恰好参与 $r-1$ 个零维持续配对:

$$(v_{k_2}, v_j), (v_{k_3}, v_j), \dots, (v_{k_r}, v_j),$$

对应持续图中的点

$$(f(v_{k_2}), f(v_j)), (f(v_{k_3}), f(v_j)), \dots, (f(v_{k_r}), f(v_j)).$$

注记 5.2 (直观理解). 当多个分量合并时, 考虑 0-闭链:

$$c_2 = v_{k_2} + v_{k_1}, \quad c_3 = v_{k_3} + v_{k_1}, \quad \dots, \quad c_r = v_{k_r} + v_{k_1}.$$

它们在 $H_0(K_{j-1})$ 中独立, 各自在 K_{k_i} 处创建, 但在进入 K_j 时全部变为平凡 (被销毁)。只有最“古老”的分量 v_{k_1} 对应的类 $c_1 = v_{k_1} + c$ 继续存活, 不参与死亡配对。

5.4 算法描述

基于 Proposition 3.17, 零维持续同调的计算无需矩阵约简, 只需维护各 K_i 的连通分量信息, 支持:

- 查询顶点所属分量的代表元;
- 合并多个分量的操作。

这正是 **并查集 (Union-Find)** 数据结构的典型应用。

算法 5.3 (ZEROPERDG). **输入:** 1-复形 $K = (V, E)$, 顶点函数 $f: V \rightarrow \mathbb{R}$

输出: 生成 $\text{Dgm}_0(f)$ 的顶点配对

1. 按 f 值升序排序顶点: $v_1, \dots, v_n$
2. 对 $j = 1$ 到 n :

(a) $\text{CREATESET}(v_j)$

(b) 令 $\text{flag} \leftarrow 0$

(c) 对 $\text{Lst}(v_j)$ 中每条边 (v_k, v_j) :

- 若 $\text{flag} = 0$, 则执行 $\text{UNION}(v_k, v_j)$ 并置 $\text{flag} \leftarrow 1$;
- 否则, 若 $\text{FINDSET}(v_k) \neq \text{FINDSET}(v_j)$, 则:
 - 令 $\ell_1 \leftarrow \text{REPSET}(v_k)$, $\ell_2 \leftarrow \text{REPSET}(v_j)$
 - $\text{UNION}(v_k, v_j)$

– 输出配对 $(\arg \max\{f(\ell_1), f(\ell_2)\}, v_j)$

3. 对每个最终的分量 C , 输出配对 $(\text{REPSET}(C), \infty)$

注记 5.4. 并查集操作说明:

- $\text{CREATESET}(x)$: 创建仅含 x 的新集合
- $\text{FINDSET}(x)$: 返回 x 所在集合的代表元
- $\text{UNION}(x, y)$: 合并 x 与 y 所在的集合
- $\text{REPSET}(x)$: 返回 x 所在集合中 f 值最小的顶点 (即代表元)

5.5 复杂度分析

- 排序顶点: $O(n \log n)$
- 并查集操作 (CREATE、FIND、UNION、REPSET) 次数: $O(n + m)$
- 总时间复杂度: $O(n \log n + m\alpha(n))$, 其中 $\alpha(n)$ 为反阿克曼函数 (增长极慢, 可视为常数)

定理 5.5 (Theorem 3.18). 对于 PL -函数 $f: |K| \rightarrow \mathbb{R}$, 零维持续图 $\text{Dgm}_0 f$ 可由 [ZEROPERDG 算法](#) 在 $O(n \log n + m\alpha(n))$ 时间内计算, 其中 n 和 m 分别为 K 的顶点数和边数。

5.6 与最小生成树的联系

若将 K 的 1-骨架视为加权图 $G = (V, E)$, 边权重设为

$$w(u, v) = \max\{f(u), f(v)\},$$

则 [ZEROPERDG 算法](#) 本质上等价于在 G 上运行 **Kruskal 最小生成树算法**。

当一条边 $e = (u, v)$ 连接两个不同分量时, 找出两个分量的最小顶点 ℓ_1, ℓ_2 , 将 e 与其中 f 值较大的那个顶点配对。然后将所有顶点-边配对 (u, e) 转换为顶点-顶点配对 (u, v) , 其中 $e \in \text{Lst}(v)$; 形如 (u, u) 的本地配对被丢弃。

5.7 图滤过情形

[ZEROPERDG 算法](#) 可直接推广到图滤过: 按滤过顺序处理顶点和边, 用并查集维护连通分量。每条连接两个不同分量的边 $e = (u, v)$, 与两个分量中较年轻的顶点配对。未配对的顶点和边分别提供零维和一维持续图中的无限条。

注记 5.6. 在图滤过情形下, 顶点排序由输入滤过隐式给出, 因此复杂度可降至 $O(n\alpha(n))$ 。